```
  end

  def state=(value)
    Test.states[object_id] = value
  end

  def state
    Test.states[object_id]
  end
end

class Client
  include Test
end

c1 = Client.new
c2 = Client.new
c1.state = "cat"
c2.state = "dog"
c1.state  # => "cat"
c2.state  # => "dog"
```

A downside of this approach is that the data associated with a particular object will not get automatically deleted if the object is deleted. In general, a mixin that requires its own state is not a mixin—it should be written as a class.

Method Lookup

Because of modules being mixed in, Ruby's story for method lookup becomes more complicated. In particular, what happens if methods with the same name are defined in a class, in that class's parent class, and in a module included into the class?

When a method is called, Ruby looks for a definition of the method. Typically, this search starts in the receiver's class. If the method is found there, great! Ruby executes that method. If not, Ruby continues up the search tree to included modules and superclasses.

The exact order of places that Ruby searches for an instance method is:

Starting with the receiver's class:

- Any module that has been added to the receiver's class using `prepend`, the last module so added is checked first.
- Methods that have specifically been added to that instance using the `def foo.bar` that we've seen or the `class << foo` syntax that we haven't talked about yet.
- Methods that are actually defined in the receiver's class.
- Any module that is added to the receivers's class using `include`, the last module so added is checked first.

If the method has not yet been found at this point, the entire loop is started over with the superclass of the receiver's class. And so on up until either a match is found or no match is found.

The entire list of classes in this lookup path can be accessed by calling the method `ancestors`, as in `String.ancestors`. Modules that have been prepended will show up before the receiver, and superclasses and appended modules will show up after.